Early Warning System & Prediction API

Internship Project — Handover & Documentation

A FastAPI backend that predicts student risk, dropout, GPA, fee defaults and admissions, and raises early-warning alerts for a Student ERP system.

Field	Detail
Prepared by	Muhammad Abdullah
Role	Backend Developer (Intern)
Module	Early Warning System & API Integration
Date	June 2026

Project links

Everything is live and available at the links below.

Resource	Address
Live API (base URL)	https://muhammadabdullah31808-early-warning-api.hf.space
Live interactive docs	https://muhammadabdullah31808-early-warning-api.hf.space/docs
Source code (GitHub)	https://github.com/muhammadabdullah901/early-warning-api
Hosting (Hugging Face Space)	https://huggingface.co/spaces/muhammadabdullah31808/early-warning-api

1. The task — what I was asked to build

My module for the Student ERP had two parts:

- **Prediction models** — predict whether a student may fail, drop out, default on fees, their future GPA, admission chances, and personalised recommendations.
- **Early Warning System** — turn those predictions into clear alerts (fail risk, low attendance, dropout risk, fee risk) and connect them with the ERP.

The deliverables were a FastAPI backend, ERP integration, and API documentation. All are complete, and the API is also deployed live.

2. What I built (summary)

- **FastAPI backend** — 6 prediction endpoints + 1 alerts endpoint, all working and tested.
- **Alerts engine** — converts prediction scores into actionable alerts with a severity level.
- **ERP integration** — one clean layer that talks to the ERP; runs on mock data now, switches to the real ERP with one setting.
- **Documentation** — interactive Swagger docs, a written reference, and this document.
- **Live deployment** — the API runs on the internet (Hugging Face) with the code on GitHub.

3. The API routes (with full addresses)

Base URL: https://muhammadabdullah31808-early-warning-api.hf.space

The full address of any route = base URL + the path below. For example, the student-risk route is at …/prediction/student-risk.

Method	Path	What it does
GET	/health	Check the server is running.
POST	/prediction/student-risk	Academic risk + whether the student may fail.
POST	/prediction/dropout	Probability the student drops out.
POST	/prediction/gpa	Predicted GPA for next term.
POST	/prediction/fee-default	Probability the fee is defaulted / paid late.

POST	/prediction/admissions	Admission probability + decision for an applicant.
POST	/prediction/recommendations	Personalised suggestions for the student.
GET	/alerts/student/{id}	All early-warning alerts for a student (reads the ERP).

Example — request and the real response from the live server

POST /prediction/student-risk — request body:

```
{
  "student_id": "STU-1001",
  "attendance_percentage": 45,
  "current_gpa": 1.6,
  "assignments_submitted": 2,
  "assignments_total": 10,
  "backlogs": 3
}
```

Response (actual output):

```
{
  "student_id": "STU-1001",
  "risk_score": 0.623,
  "risk_level": "high",
  "may_fail": true,
  "alerts": [
    { "type": "may_fail", "severity": "high",
      "message": "High academic risk (score 0.62). Student may fail." },
    { "type": "low_attendance", "severity": "high",
      "message": "Attendance is 45%, below the required 75%." }
  ]
}
```

4. How a frontend developer uses this API

The API is a standard REST + JSON service, so any frontend (React, Angular, Vue, plain JavaScript or mobile) can call it. No API key is needed right now, and CORS is enabled so it works directly from the browser. Three easy ways:

- **1. In the browser** — open the live docs (…/docs), press 'Try it out' on any route, edit the data, press Execute. The response shows instantly.
- **2. From code** — send a normal HTTP request from the app (example below).
- **3. Quick check** — open a GET route (…/health, …/alerts/student/STU-1001) directly in a browser.

JavaScript example (fetch):

```
const res = await fetch(
  'https://muhammadabdullah31808-early-warning-api.hf.space/prediction/student-risk',
  {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      student_id: 'STU-1001', attendance_percentage: 45, current_gpa: 1.6,
      assignments_submitted: 2, assignments_total: 10, backlogs: 3
    })
  }
);
const data = await res.json();
console.log(data.risk_level, data.may_fail, data.alerts);
```

The frontend reads the JSON response (risk_score, risk_level, may_fail, alerts) and shows it on screen. Every route's exact fields are listed on the …/docs page, so the frontend developer never has to guess.

5. The Early Warning System

After a prediction is calculated, the alert engine checks it against sensible thresholds and produces alerts. Each alert has a type, a severity (low / medium / high) and a clear message. The four alert types are:

- **Student may fail** — academic risk is high enough that the student may fail.
- **Low attendance** — attendance is below the required 75%.
- **Dropout risk** — the dropout probability is high.
- **Fee payment risk** — the student is likely to pay late or default.

The GET /alerts/student/{id} route shows the full flow: it reads the student's record from the ERP, runs the checks, and returns every alert that applies (and pushes them back to the ERP in a real setup).

6. ERP integration

All ERP communication lives in one file (app/integrations/erp_client.py), with two modes controlled by the ERP_MOCK_MODE setting:

- **Mock mode** — the default; returns realistic sample students so the system works without the real ERP. This is what the live demo uses.

- **Real mode** — set ERP_MOCK_MODE=false and add the ERP URL + API key; the same code then talks to the real ERP. No other change needed.

7. How the code is organised

I used a layered structure so any developer can navigate it quickly — each folder has one job:

Folder	Responsibility
app/api/routes/	The endpoints — receive requests, return responses.
app/schemas/	Data shapes with automatic validation.
app/services/	Business logic — prediction and alert rules.
app/models/	ML model placeholder — swap in a real model here later.
app/integrations/	Talks to the ERP (mock or real).
tests/	Automated tests for every endpoint.

8. How to run it locally

Python 3.10 or newer is required.

```
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Then open http://127.0.0.1:8000/docs. Run the tests with: pytest

9. Deployment

The code is on GitHub and the live API runs on Hugging Face Spaces (using a Dockerfile, free plan, no card required). Each time new code is pushed, Hugging Face rebuilds and redeploys automatically. On the free plan the Space sleeps after long inactivity and wakes up in about half a minute on the next request — normal for free hosting.

10. Testing

The repository includes TESTING.md with four ready sample inputs for every route. Four sample students are already loaded for the alerts route:

Student ID	Profile	What you see
STU-1001	Ali Khan — at risk	Fail + low-attendance alerts
STU-1002	Sara Ahmed — healthy	No alerts
STU-1003	Bilal Hussain — medium	Attendance, dropout & fee alerts
STU-1004	Hina Raza — high risk	Fail, attendance, dropout & fee alerts